SDK - HTHardDll.dll 说明文档

中文版(VC++ 6.0)

阅读须知：

本 DLL 在 VC++ 6.0 环境下编译生成。所以数据类型符合 VC++ 6.0 标准.

WORD ：unsigned short，无符号 16bit 整型。

BOOL ： bool，布尔类型。

ULONG：unsigned long，无符号 32bit 整型。

unsigned char：无符号 8 位整型

short：有符号 16 位整型

此 DLL 中的所有文件都是用命令行上定义的 DLL_API 符号编译的。在使用此 DLL 的任何其他项目上都不应定义此符号。这样，源文件中包含此文件的任何其他项目都会将 DLL_API 函数视为是从 DLL 导入的。

```
#ifndef    DLL_API
#define    DLL_API    extern "C" __declspec(dllimport)
#endif
```

定义标准调用：

```
#define WIN_API __stdcall
```

结构体介绍

结构体 _HT_RELAY_CONTROL 包含了所有控制继电器状态所需要的信息。

```
typedef struct _HT_RELAY_CONTROL
{
    BOOL bCHEnable[4];
    WORD nCHVoltDIV[4];
    WORD nCHCoupling[4];
    BOOL bCHBWLimit[4];
    WORD nTrigSource;
    BOOL bTrigFilt;
    WORD nALT;
}RELAYCONTROL,*PRELAYCONTROL;
```

说明：

bCHEnable[4]：大小为 4(CH 的总数)的数组，表示 CH 的开/关。取值：1 为开；0 为关。

nCHVoltDIV[4]：大小为 4(CH 的总数)的数组，表示 CH 的电压档位。电压档位以索引值形式表示。以最小电压档位为 0 开始依次递加 1 计算。

nCHCoupling[4]：大小为 4(CH 的总数)的数组，表示 CH 的耦合。耦合以索引值形式表示。取值：DC 为 0；AC 为 1；GND 为 2；

bCHBWLimit[4]：大小为 4(CH 的总数)的数组，表示 CH 的带宽限制。取值：1 为打开带宽限制；0 为关闭带宽限制。

nTrigSource：表示触发源，以索引值形式取值。假设现在为 4CH 示波器，则内部触发取值为：CH1 为 0；CH2 为 1；CH3 为 2；CH4 为 3；如果有外部触发，则 EXT 为 5；如果有 EXT/10 触发，则取值为 6。

bTrigFilt：表示高频抑制。取值：1 表示打开高频抑制，0 表示关闭高频抑制。

nALT：表示是否交替。取值：1 为交替，0 为非交替。

举例：

声明一个变量：RELAYCONTROL　myRelayControl;

声明一个指针：PRELAYCONTROL　pRelayControl;

结构体 _HT_CONTROL_DATA 包含了某些函数需要的一些控制信息。

```
typedef struct _HT_CONTROL_DATA
{
        WORD nCHSet;//CH 开关---//第 0 位：表示 CH1 开或者关. 0:关，1 开
                                //第 1 位：表示 CH2 开或者关. 0:关，1 开
                                //第 2 位：表示 CH3 开或者关. 0:关，1 开
                                //第 3 位：表示 CH4 开或者关. 0:关，1 开
        WORD nTimeDIV;          //时基
        WORD nTriggerSource;    //触发源
        WORD nHTriggerPos;      //水平触发位置
        WORD nVTriggerPos;      //垂直触发位置
        WORD nTriggerSlope;     //边沿触发触发沿
        ULONG nBufferLen;       //内存长度
        ULONG nReadDataLen;     //读取数据长度
        WORD nALT;              //是否交替

}CONTROLDATA,*PCONTROLDATA;
```

举例：

声明一个变量：CONTROLDATA　myControlData;

声明一个指针：PCONTROLDATA　pControlData;

函数介绍

1. **函数声明:** DLL_API BOOL WINAPI dsoSetUSBBus (WORD nDeviceIndex)

返回值：

返回 FALSE 表示失败，返回 TRUE 表示成功

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

备注：

设置 USB 通讯总线。切换到 USB 通讯模式后，这是必须第一个被调用的操作硬件函数

2. 函数声明: DLL_API WORD WINAPI dsoHTReadCalibrationData (WORD nDeviceIndex,
WORD* pLevel,
WORD nLen
)

返回值:

返回 0 表示失败，返回 1 表示成功

参数:

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

pLevel nType

WORD 型指针变量。指向外部缓冲区，保存设备的校对值。

nLen

WORD 型变量，表示缓冲区的大小，具体值见代码。

备注:

每次初始化设备，都要首先读取设备的校对值，以确保通道的零电平位置正确。

3. 函数声明: DLL_API WORD WINAPI dsoHTSearchDevice(short* pDevInfo)

返回值:

返回连接的设备总数。1 台 PC 支持最大连接 64 台设备。

参数:

pDevInfo

short 型数组指针，用于存储有无设备信息。数组大小为 64。

备注:

获取 PC 已经连接上的设备总数，总数不超过 64。

程序举例:

```
short DevInfo[64];
WORD nConnectedDevNum = 0;
//DevInfo 初始化
//…
//调用函数
nConnectedDevNum = dsoHTSearchDevice(DevInfo);
```

如果 DevInfo[n] = 0 有设备，DevInfo[n] = -1 无设备.

4. 函数声明: DLL_API WORD WINAPI dsoHTSetCHPos(
WORD nDeviceIndex,
WORD* pLevel,
WORD nVoltDIV,
WORD nPos,
WORD nCH
)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

pLevel

WORD 型变量指针，指向校对电平。

nVoltDIV

WORD 型变量，表示电压档位索引值。

nPos

WORD 型变量，表示 CH 零电平（参考电平）位置，取值范围 0 ~ 255（8bit 精度）。

nCH

WORD 型变量，表示当前设置的 CH。CH1 为 0，CH2 为 1，CH3 为 2，CH4 为 3。

备注：

当零电平（参考电平）发生变化时，调用此函数进行设置。

程序举例:

假设要将 CH1 的零电平设置为 128。

```
WORD nDeviceIndex = 0;
WORD CHLevel[256]; //大小最小为 256，通过 dsoHTReadCalibrationData 附值。
WORD nVoltDIV = 6; //假设 1V/div 的索引值为 6。
WORD nPos = 128; //零电平 128。
WORD nCH = 0; //CH1。
//调用函数
if ( 0 = = dsoHTSetCHPos(nDeviceIndex,CHLevel,nVoltDIV,nPos,nCH) )
    ;//失败
else
    ;//成功
```

5. 函数声明：

```
DLL_API WORD WINAPI dsoHTSetVTriggerLevel(
                        WORD nDeviceIndex,
                        WORD* pLevel,
                        WORD nPos
                        )
```

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

pLevel

WORD 型变量指针，指向校对电平。

nPos

WORD 型变量，表示垂直触发电平位置，取值范围 0 ~ 255（8bit 精度）。

备注：

当需要设置触发电平时，调用此函数设置。

程序举例:

```
WORD nDeviceIndex = 0;
WORD CHLevel[256]; //大小最小为 256，通过 dsoHTReadCalibrationData 附值。
WORD nPos = 128; //触发电平为 128
//调用函数
if ( 0 = = dsoHTSetVTriggerLevel( nDeviceIndex, CHLevel, nPos) )
        ; //失败
else
        ; //成功
```

6. 函数声明：DLL_API WORD WINAPI dsoHTSetHTriggerLength(

```
WORD nDeviceIndex,
ULONG nBufferLen,
WORD HTriggerPos,
WORD nTimeDIV，
,WORD nYTFormat
)
```

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

nBufferLen

ULONG 型变量，表示内存长度。

HTriggerPos

WORD 型变量，表示水平触发位置。取值返回 0 ~ 100 。

nTimeDIV

WORD 型变量，表示时基的索引值，以时间最短时基为 0 依次递加 1 计算。

nYTFormat

WORD 型变量，3 种采样格式。0: Normal, 1: Scan, 2: Roll

备注：

设置触发和预触发长度。

程序举例:

假设当前设备内存为 10K，水平触发位置在 50%，时基为 40us（索引值为 12）。

```
WORD nDeviceIndex = 0;
ULONG nBufferLen = 10240;//10K
WORD HTriggerPos = 50; //50%
WORD nTimeDIV = 12; //40us
//调用函数
if ( 0 = = dsoHTSetHTriggerLength(nDeviceIndex,nBufferLen,HTriggerPos,
                                  nTimeDIV) )
    ; //失败
else
```

; //成功

7. 函数声明：DLL_API WORD WINAPI dsoHTSetCHAndTrigger(
WORD nDeviceIndex,
RELAYCONTROL RelayControl
)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

RelayControl

结构体，具体见结构体说明部分。

备注：

设置 CH 和 Trigger

程序举例:

```
WORD nDeviceIndex = 0;
RELAYCONTROL RelayControl;
//RelayControl 各变量附值
//调用函数
if ( 0 = = dsoHTSetCHAndTrigger(nDeviceIndex,RelayControl) )
    ; //失败
else
    ; //成功
```

8. 函数声明：DLL_API WORD WINAPI dsoHTSetCHAndTriggerVB(
WORD nDeviceIndex,
WORD* pCHEnable,
WORD* pCHVoltDIV,
WORD* pCHCoupling,
WORD* pCHBWLimit,
WORD nTriggerSource,
WORD nTriggerFilt,
WORD nALT
)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

pCHEnable

WORD 指针类型，用于传递 RELAYCONTROL 结构体中的开关部分

pCHVoltDIV

WORD 指针类型，用于传递 RELAYCONTROL 结构体中的电压档位部分

pCHCoupling

WORD 指针类型，用于传递 RELAYCONTROL 结构体中的耦合部分

pCHBWLimit

WORD 指针类型，用于传递 RELAYCONTROL 结构体中的带宽限制部分

nTriggerSource

WORD 类型，用于传递 RELAYCONTROL 结构体的触发源部分

nTriggerFilt

WORD 类型，用于传递 RELAYCONTROL 结构体中的高频抑制部分

nALT

WORD 类型，用于传递 RELAYCONTROL 结构体中的是否交替部分

备注：

设置 CH 和 Trigger，该接口用于 VB 开发环境

程序举例:

```
WORD nDeviceIndex = 0;
WORD pCHEnable[4] = {1,1,1,1};
WORD pCHVoltDIV[4] = {6,6,6,6};
WORD pCHCoupling[4] = {1,1,1,1};
WORD pCHBWLimit[4] = {0,0,0,0};
WORD nTriggerSource = 0; //CH1
WORD nTriggerFilt = 0;
WORD nALT = 0;
//调用函数
if(0==dsoHTSetCHAndTriggerVB(nDeviceIndex,pCHEnable,pCHVoltDIV, pCHCoupling,
    pCHBWLimit, nTriggerSource, nTriggerFilt, nALT) )
    ; //失败
else
    ; //成功
```

9. 函数声明：

```
DLL_API WORD WINAPI dsoHTSetTriggerAndSyncOutput(
                                    WORD nDeviceIndex,
                                    WORD nSynMode,
                                    WORD nlaTriggerMode,
                                    WORD nTriggerMode,
                                    WORD nTriggerSlope,
                                    WORD nPWCondition,
                                    ULONG nPW,
                                    USHORT nVideoStandard,
                                    USHORT nVedioSyncSelect,
                                    USHORT nVideoHsyncNumOption,
                                    WORD nSync
```

)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

nSynMode

WORD 型变量，用于选择触发源类型

取值：

0x00 ： 当选择触发源 CH1、CH2、EXT

0x05 ： 当选择触发源 D0~D17

nlaTriggerMode

WORD 型变量，表示触发源 D0~D17 的触发类型。取值：边沿触发为 0，

nTriggerMode

WORD 型变量，表示触发源 CH1、CH2、外触发的触发类型。取值：边沿触发为 0，脉冲触发为 1。

nTriggerSlope

WORD 型变量，表示上升沿/下降沿。取值：上升沿为 0，下降沿为 1。

nPWCondition

WORD 型变量，表示脉冲触发条件。（脉冲触发下有效）

取值：

+ Less 为 0 ： 当正脉宽 小于 nPW 时进行触发。

+ Equal 为 1 ： 当正脉宽 等于 nPW 时进行触发。

+ More 为 2 ： 当正脉宽 大于 nPW 时进行触发。

- Less 为 3 ： 当负脉宽 小于 nPW 时进行触发。

- Equal 为 4 ： 当负脉宽 等于 nPW 时进行触发。

- More 为 5 ： 当负脉宽 大于 nPW 时进行触发。

nPW

ULONG 型变量，表示脉冲宽度设置。脉冲宽度时间范围为 10ns ~ 10s。nPW 的取值范围为 2 ~ 2000000000（10ns/5 ~ 10000000000ns/5）。（脉冲触发下有效）

nVideoStandard

USHORT 变量，表示视频标准设置。取值 0 为 PAL/SECAM ,1 为 NTSC

nVedioSyncSelect

USHORT 变量,表示同步选项设置。取值：

所有行：0

所有场：1

奇数场：2

偶数场；3

指定行：4

nVideoHsyncNumOption

USHORT 变量，表示指定行数设置。当同步参数（nVedioSyncSelect）设置为指定行时有效。最小值为 1。

nSync

WORD 型变量，表示同步输出。取值：0 关闭同步输出，1 打开同步输出。

备注：

设置 Trigger 和同步输出。如果是边沿触发，参数 nPWCondition 和 nPW 无效。

程序举例:

```
WORD nDeviceIndex = 0;
WORD nSynMode = 0;//表示触发 CH1 或 CH2
WORD nlaTriggerMode = 0; //当前是边沿触发。
WORD nTriggerMode = 0; //当前是边沿触发。
WORD nTriggerSlope = 0; //当前是上升沿触发。
WORD nPWCondition = 0; //
ULONG nPW = 2; //10ns, nPW = 10ns/5。
WORD nSync = 0; //关闭同步输出。
//调用函数
if (0 = = dsoHTSetTriggerAndSyncOutput(nDeviceIndex, nSynMode , nlaTriggerMode,
                    nTriggerMode, nTriggerSlope, nTriggerCondition, nPW,nSync) )
    ; //失败
else
    ; //成功
```

10. 函数声明：DLL_API WORD WINAPI dsoHTSetSampleRate(
WORD nDeviceIndex,
WORD nTimeDIV,
WORD nYTFormat
)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

nTimeDIV

WORD 型变量，表示时基的索引值，以时间最短时基为 0 依次递加 1 计算,

nYTFormat

WORD 型变量，3 种采样格式。0: Normal, 1: Scan, 2: Roll

备注：

设置时基（采样率）。

程序举例:

如果我们想要把时基设置到 40us.

```
WORD nDeviceIndex = 0;
WORD nTimeDIV = 12; //假设 40us 的索引值为 12。
//调用程序
if (0 = = dsoHTSetSampleRate(nDeviceIndex, nTimeDIV) )
    ; //失败
else
    ; //成功
```

11. 函数声明：DLL_API WORD WINAPI dsoHTStartCollectData(WORD nDeviceIndex)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

备注：

通知设备准备采集数据。当各种设置都设备完毕，进入采集数据状态时，第 1 步就是调用此函数通知设备，准备采集数据。这里要注意，调用此函数之后，只有在设备采集完本次数据后，且进入新一轮数据采集时，才能再次调用，也就是采集一次数据只能调用一次本函数。

程序举例:

```
WORD nDeviceIndex = 0;
if (0 = = dsoHTStartCollectData(nDeviceIndex) )
    ; //失败
else
    ; //成功
```

12. 函数声明：DLL_API WORD WINAPI dsoHTStartTrigger(WORD nDeviceIndex)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

备注：

通知设备准备检测触发信号。在采集状态下，调用函数 dsoHTStartCollectData 成功之后，调用本函数启动触发信号检测。

程序举例:

```
WORD nDeviceIndex = 0;
if (0 = = dsoHTStartTrigger(nDeviceIndex) )
    ; //失败
else
    ; //成功
```

13. 函数声明：DLL_API WORD WINAPI dsoHTForceTrigger(WORD nDeviceIndex)

返回值：失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

备注：

调用此函数可以进行一次强制触发采集数据。触发点随机确定。

程序举例:

```
WORD nDeviceIndex = 0;
if (0 = = dsoHTForceTrigger(nDeviceIndex) )
    ; //失败
else
    ; //成功
```

14. 函数声明：DLL_API WORD WINAPI dsoHTGetState(WORD nDeviceIndex)

返回值：

当返回值为 0x07 时，表示设备已经采集完数据，可以进行读取数据。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

备注：

判断设备是否采集完数据。

程序举例:

```
WORD nDeviceIndex = 0;
while ( dsoHTGeState(nDeviceIndex) != 0x07)
{
    continue;
}
//读取数据
//……..
```

15. 函数声明：DLL_API WORD WINAPI dsoHTGetData (

```
WORD nDeviceIndex,
WORD* pCH1Data,
WORD* pCH2Data,
WORD* pCH3Data,
WORD* pCH4Data,
PCONTROLDATA pControl
);
```

返回值：

失败返回 0，成功返回非 0。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

pCH1Data

WORD 型指针变量。指向保存 CH1 数据的缓冲区的指针。

pCH2Data

WORD 型指针变量。指向保存 CH2 数据的缓冲区的指针。

pCH3Data

WORD 型指针变量。指向保存 CH3 数据的缓冲区的指针。

pCH4Data

WORD 型指针变量。指向保存 CH4 数据的缓冲区的指针。

pControl

见结构体说明部分。

备注：

读取数据

程序举例:

```
WORD nDeviceIndex = 0;
if ( 0 = = dsoHTGetData ( nDeviceIndex, pCH1Data, pCH2Data, pCH3Data, pCH4Data
                          , pControl) )
    ; //读取数据失败
else
    ; //读取数据成功
```

16. 函数声明：DLL_API BOOL WINAPI laSetTrigSource(
WORD DeviceIndex,
unsigned char nSour
)

返回值：

失败返回 false，成功返回 true。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

nSour

unsigned char 型变量,表示触发逻辑通道

取值：

0 到 15：对应于 D0~D15。

备注：

当选者逻辑通道时，触发对应的逻辑通道 D0 到 D15

程序举例:

```
WORD nDeviceIndex = 0;
unsigned char   nSour  =  0; //触发通道 D0
if ( false = = laSetTrigSource(DeviceIndex, nSour))
    ; //失败
else
```

; //成功

17. 函数声明：DLL_API BOOL WINAPI laSetVoltDiv(
WORD DeviceIndex,
WORD *nlaVoltDiv
)

返回值：

失败返回 false，成功返回 true。

参数：

nDeviceIndex

WORD 型变量，表示当前设备的索引值。

nlaVoltDiv

WORD 指针型变量,指向 16 个逻辑通道的电压档位，

取值：

为 0~119：对应于-6V~+6V

备注：

设定逻辑通道门电平值

程序举例:

```
WORD nDeviceIndex = 0;
WORD nlaVoltDiv[16];
WORD i;
for(i=0;i<16;i++)
{
    nlaVoltDiv[i] = 58;//-0.2V
}
if ( false = = aSetVoltDiv(WORD DeviceIndex, nlaVoltDiv))
    ; //失败
else
    ; /成功
```

18. 函数声明：DLL_API BOOL WINAPI dsoGetDisplayDataFromSourceDataSingle(
short *pDisplayData,
WORD *pSourceData,
WORD *LeverPos,
ULONG nDataLen,
WORD nCH
)

返回值：

失败返回 false，成功返回 true。

参数：

pDisplayData

short 指针型变量，指向数据数组。

pSourceData,

WORD 指针型变量，指向数据数组。

LeverPos,

WORD 指针型变量，指向 18 个通道屏幕显示位置的数据数组。

nDataLen

ULONG 型变量，表示通道数据长度，默认值为 10240

nCH

WORD 型变量，表示 18 个通道中的某一个，取值 0~17

备注：

用于从 4 个通道数据中，分离出来 18 个通道

其中 CH1、CH2 位置不变，由 CH3 分离出 D0~D7，由 CH4 分离出 D8~D15

程序举例：

```
short pDisplayData[18][10240];    //18 个通道数据
WORD pSourceData[4][10240];   //采集上来的四个通道数据
WORD LeverPos[18];                //18 个通道显示位置
ULONG nDataLen = 10240;           //默认值
WORD nCH;                         //通道
    for (i=0;i<18;i++)
    {
        if(i<2)
        {
            if(!(dsoGetDisplayDataFromSourceDataSingle(pDisplayData[i],
                pSourceData[i] , LeverPos, nDataLen,i)))
            {
                //数据处理出错
                break;
            }
        }
        else if(i<10)
        {
            if(!(dsoGetDisplayDataFromSourceDataSingle(pDisplayData[i],
                pSourceData[2] , LeverPos, nDataLen,i)))
            {
                //数据处理出错
                break;
            }
        }
        else if(i<18)
        {
            if(!(dsoGetDisplayDataFromSourceDataSingle(pDisplayData[i],
                pSourceData[3] , LeverPos, nDataLen,i)))
            {
```

```
                    //数据处理出错
                    break;
                }
            }
        }
        if (i == 18)
        {
            //数据处理成功
        }
```

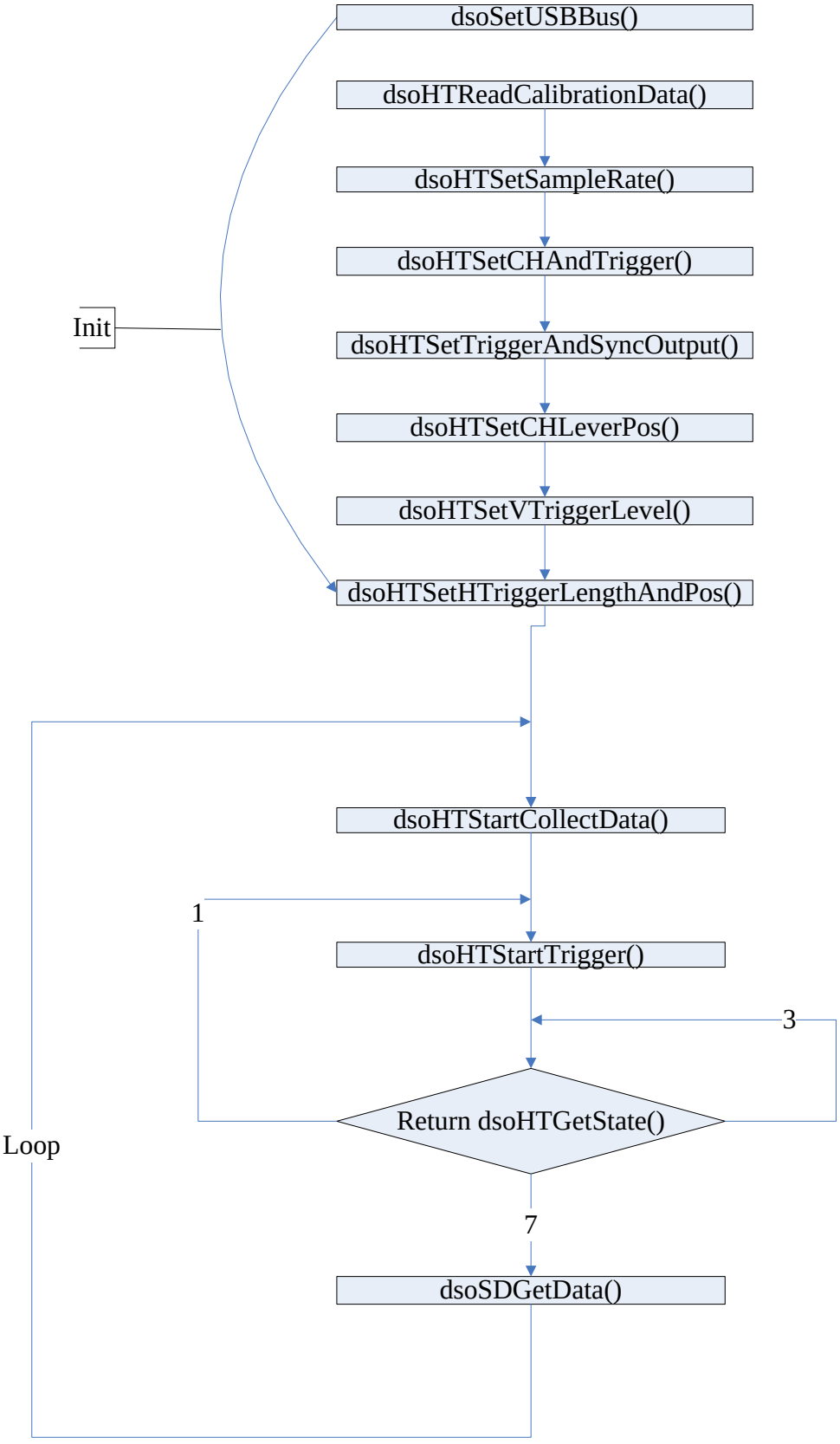
备注：

获取 CH1、CH2 以及 D0~D15 单个通道数据

程序举例：

```
WORD nDeviceIndex = 0;
unsigned char    nSour  ＝  0; //触发通道 D0
if (false = = laSetTrigSource(DeviceIndex, nSour))
    ; //读取数据失败
else
    ; //读取数据成功
```

控制流程图如下：


dsoSetUSBBus()
dsoHTReadCalibrationData()
dsoHTSetSampleRate()
dsoHTSetCHAndTrigger()
Init
dsoHTSetTriggerAndSyncOutput()
dsoHTSetCHLeverPos()
dsoHTSetVTriggerLevel()
dsoHTSetHTriggerLengthAndPos()
dsoHTStartCollectData()
1
dsoHTStartTrigger()
3
Return dsoHTGetState()
Loop
7
dsoSDGetData()